Welcome to your next lesson: Functions i: why, define, and call.

I'm preparing your personalized lesson based on what you've learned, what comes next, and the best way to reinforce your skills.

Step by step, you're moving forward.

-

Imagine you're writing a program that says hello to different people.

Without functions, you might do this:

```python
print("Hello, Alice")
print("Hello, Bob")
print("Hello, Carol")
```

You're repeating the same pattern three times — only the name changes. Functions are a way to package that pattern once and then reuse it whenever you want, with different names.

Quick check for you, Ron:

If you had to add 50 more names, which sounds better:

- keep copying more `print("Hello, ...")` lines, or
- write the pattern once and reuse it?

Just answer in your own words which approach feels better and why.

Writing the pattern once feels easier

Exactly ?

That "write the pattern once" idea is the whole reason functions exist.

Let me show you the same greeting example with a function:

```python
def greet(name):
    print("Hello, " + name)

greet("Alice")
greet("Bob")
greet("Carol")
```

Step by step:

1. `def greet(name)` → this defines a function called `greet` that expects a `name`.
2. The indented line runs whenever you call `greet(...)`.
3. `greet("Alice")` uses the function instead of re-writing the `print` code.

So now the "Hello, ___" pattern lives in one place.

Make sense so far: a function is like saving a pattern and then reusing it by calling its name?

You can just answer yes/no, or tell me what part feels fuzzy.

Yes, that makes sense

Great ?

Now let's zoom in on the shape of a function in Python — its basic structure.

Here's a simple one:

```Python
def greet(name):
    print("Hello, " + name)
```

Visually, it has 4 key parts:

1. **def** → a keyword that tells Python: "I'm defining a function now."
2. greet → the function name (you choose this, like a variable name).
3. (name) → parentheses with parameters (placeholders for values you'll pass in).
4. and the indented line below → the function's body (the code that runs).

You can think of it like:

```Text
def   name   (parameters):
    # body (indented)
```

Does this 4-part structure feel clear, or is there any part (like parameters or the colon/indent) you want me to slow down on?

makes sense i wsa going to add that that wasnt i your description that besides def you need the colon after the closing paren before ident but oyu covered it

Nice catch on the colon and indent — that attention to detail will help a lot in Python ?

Next, I want to show you clearly how defining a function and calling it are two separate steps.

Definition (save the pattern):

```Python
def greet(name):
    print("Hello, " + name)
```

Call (actually use it):

```Python
greet("Ron")
```

Nothing happens when you only define the function — Python just stores the pattern.

Something happens (it prints) only when you call **greet**("Ron").

Does that difference between "define" vs "call" feel clear?

If it's a bit fuzzy, tell me which part: the **def** part, or the **greet**("Ron") part.

It's clear

Perfect. Let's make this more concrete with a money example ?

Imagine this code without a function:

```Python
price1 = 10
tax_rate = 0.08
```

```
print("Total:", price1 + price1 * tax_rate)

price2 = 25
print("Total:", price2 + price2 * tax_rate)

price3 = 40
print("Total:", price3 + price3 * tax_rate)
```

Same pattern repeated 3 times: `price + price * tax_rate`.

We can turn that into a function:

Python

```
def print_total_with_tax(price, tax_rate):
    total = price + price * tax_rate
    print("Total:", total)

print_total_with_tax(10, 0.08)
print_total_with_tax(25, 0.08)
print_total_with_tax(40, 0.08)
```

Now:

- The formula lives once inside `print_total_with_tax`.
- We just reuse it with different numbers.

Step check ?

Do you see how the second version reduces repetition compared to the first? If yes, I'll have you write a tiny function next in the editor.

> yes it makese sense i have quick question about sope withi functionns thouht i would say its bad prqactice if there were a varialbe price = 5 o the first liine of th progra, but then in def total_funnctio(price, tax_rate): print (price * ta_rate)
> total_function(price + 2, tax_rate)
> is the price inside the function allowe and does it isokay 5 (sees the ovfirtt price thats still 5, or it ow local copy inside the function thats 7 as it waspassed 5 = 2

Love that you're thinking about this already — that's a more advanced question ?

But for this lesson, we actually need to stay away from scope topics (local vs global), so I can't dive into that fully yet.

For now, use this simple mental model:

> *When you call a function like* `total_function(price + 2, tax_rate)`,
> *Python calculates the value first, then hands that value into the function.*

So in your example:

Python

```
price = 5
```

```
def total_function(price, tax_rate):
    print(price * tax_rate)

total_function(price + 2, tax_rate)
```

- Outside, price is 5.
- price + 2 is 7.
- The function receives 7 for its price parameter and uses that value.

Later, when we officially cover scope, we'll talk about why the inside-price is its own thing.

Want to try a small function in the editor now?

Something like a greet(name) or print_total_with_tax(price, tax_rate) that you define and then call twice.

sure

Awesome ?

Let's do this step by step.

1. In the code editor, write a function that prints a greeting to a person:
   - It should be named greet
   - It should take one parameter (call it name)
   - Inside, it should print something like: Hello, <name>!
2. After the function definition, call it twice:
   - once with a literal, like "Ron"
   - once using a variable, like:

```
person = "Alice"
greet(person)
```

3. Hit Run to make sure it works, then hit Submit code so I can see it.

Don't worry about being perfect — write your best version in the editor, run it, then submit the code and I'll give you feedback ?

Python

```
name = "Ron"
def greet(greet_name):
    print("Hello, " + greet_name)
greet(name)
greet("Alice")
```

Nice work, Ron ?

You did exactly what I asked:

Python

```
name = "Ron"
```

```
def greet(greet_name):
    print("Hello, " + greet_name)

greet(name)
greet("Alice")
```

- You defined a function `greet(greet_name)` with one parameter.
- You called it once with a variable (`name`) and once with a literal (`"Alice"`).
- Output is:
- Hello, Ron
- Hello, Alice

That's perfect.

Next step: let's do a two-parameter function (like your tax example), so you get used to multiple inputs.

I'll explain that in the next message — ready to try a function with two parameters after I walk through one example?

Yes, I'm ready

Great ? Let's keep it simple and visual.

Here's a two-parameter function:

Python

```
def print_total(price, quantity):
    total = price * quantity
    print("Total cost:", total)

# calls:
print_total(5, 3)      # 5 dollars, 3 items
amount = 12
count = 2
print_total(amount, count)
```

Step by step:

- `def print_total(price, quantity):` → defines the function with two parameters.
- Inside, `total = price * quantity` is the pattern.
- `print_total(5, 3)` → passes `5` and `3` directly (literals).
- `print_total(amount, count)` → passes values stored in variables.

Your turn in the editor ?

1. Define a function named `compute_total(item_price, quantity)` that:
- multiplies `item_price * quantity`
- prints the result (you can include text like `"Total:"` if you want).

2. Call it twice:

- once with literals (like `compute_total(4, 3)`)
- once with variables (like `price = 10`, `qty = 2`, then `compute_total(price, qty)`).

Run it, then hit Submit code, and I'll review it.

```python
price = 6
quantity = 4
def compute_total(item_price, quantity):
    print("Total " + str(item_price * quantity))
compute_total(4, 3)
compute_total(price, quantity)
```

This is excellent, Ron ?

```python
price = 6
quantity = 4

def compute_total(item_price, quantity):
    print("Total " + str(item_price * quantity))

compute_total(4, 3)
compute_total(price, quantity)
```

You nailed the goals:

- Function with two parameters: `item_price` and `quantity`.
- Inside, you compute `item_price * quantity` and print it.
- You called it:
- once with literals → `compute_total(4, 3)` → `Total 12`
- once with variables → `compute_total(price, quantity)` → `Total 24`

That covers all the main skills for this lesson: why functions matter, their structure, and calling them with literals and variables ?

Before we wrap, I want to quickly check your understanding of the shape of a two-parameter function.

What do you think about trying a quick one-question multiple choice check?

Sure, let's try!

What do you think? Pick the option that shows a correctly written Python function ?

Which of the following correctly defines a Python function with two parameters that prints their product?

```
# Option A
multiply(a, b):
```

```
        print(a * b)
# Option B
def multiply(a, b)
        print(a * b)
# Option C
def multiply(a, b):
print(a * b)
# Option D
def multiply(a, b):
        print(a * b)
```

A.

Option A

B.

Option B

C.

Option C

D.

Option D

Nice job, Ron ? You picked Option D, which is exactly right:

Python

```
def multiply(a, b):
        print(a * b)
```

Why it’s correct, step by step:

- Starts with `def` ✅
- Has a name `multiply` ✅
- Two parameters in parentheses: `(a, b)` ✅
- A colon `:` at the end of the `def` line ✅
- The body (`print(a * b)`) is indented ✅

The other options each miss one of those pieces (no `def`, missing colon, or no indentation).

You’ve shown you can:

- Explain why functions are useful (less repetition, more reuse)
- Read and write function definitions with 1–2 parameters
- Call functions with both literals and variables

You’ve basically mastered everything in this lesson ?

Want to wrap up this functions lesson and move on to the next one, or would you like one more small practice challenge first?

I'm ready for the next lesson

Awesome work today, Ron ?

Quick recap of what you just mastered:

- You saw why functions matter: write a pattern once, reuse it many times instead of repeating code.
- You learned the structure of a Python function:
- `def` + name + `(parameters)` + `:`
- indented body that runs when you call it.
- You defined and called your own functions:
- `greet(name)` with one parameter
- `compute_total(item_price, quantity)` with two parameters
- and you called them with both literals and variables.

This is a big step: you've moved from just writing straight-line code to creating reusable building blocks ?

You can hit Next Lesson in the Learn tab whenever you're ready, and I'll keep building on this with you.

If you want to sneak in any quick question about functions before you go, drop it here and I'll help.

Thank you for your feedback!